Gross Ahead: Final Project for the Deep Learning Course

Yuval Dreizin, Tomer Ohayon

1 Introduction

In this project, we study the problem of predicting a movie's box-office gross using only metadata features. The available information is tabular and heterogeneous, combining numerical attributes such as budget and release year with categorical descriptors such as genre, director, cast, production company, and country. This setting provides a practical testbed for exploring how different neural network architectures and training strategies handle structured data with diverse semantic components.

We begin by constructing a deep neural network baseline for movie gross prediction and then investigate whether introducing explicit structure into the model can improve performance. In particular, we examine architectures that separate input features into semantically motivated groups and compare them to a standard shared-encoding approach. By varying how and when information from different feature groups is combined, we aim to better understand the role of feature interaction in tabular regression models.

In addition to architectural design, we focus on the challenge of temporal distribution shift. Movie industry data evolves over time, and models trained on historical data may not generalize well to more recent samples. To study this setting, we adopt a pretrain–fine-tune framework in which a model is first trained on older movies and then adapted to later ones. Within this framework, we compare several fine-tuning strategies that differ in the number of trainable parameters and the extent of representation adaptation, particularly under limited data and training budget constraints.

2 Dataset and Preprocessing

We use the Kaggle Movie Industry dataset, containing metadata for movies released over multiple decades. After cleaning and filtering, the final dataset consists of 5,435 samples, split into training, validation, and test sets using a fixed 70/15/15 split.

Numerical features (e.g., budget, runtime, release year, votes) are normalized. Categorical features (e.g., genre, director, actor, company, country, rating) exhibit long-tailed distributions; categories with at most five occurrences are mapped to a shared *other* token. All categorical features are encoded using learned embeddings.

3 Baseline Model

3.1 Architecture

As a reference point for all subsequent experiments, we implement a fully connected deep neural network for tabular regression. The model processes all numerical and categorical features jointly through a shared encoder, without imposing any explicit structure or separation between feature groups.

Numerical features are provided directly as inputs, while categorical features are first transformed into learned embedding vectors and then concatenated with the numerical inputs. The resulting feature vector is passed through a stack of fully connected layers with ReLU activations. Dropout with probability 0.1 is applied between layers. The final layer outputs a single scalar corresponding to the predicted movie's gross.

3.2 Training Setup

The model is trained using the AdamW optimizer with a learning rate of 3×10^{-3} and a weight decay of 1×10^{-4}. MSE is used as the training objective. A ReduceLROnPlateau scheduler is used to adjust the learning rate based on validation performance. Early stopping based on validation loss is applied during training.

All training and evaluation procedures use the same data splits described in Section 2.

3.3 Hyperparameter Selection and Parameter Count

The baseline architecture, including the number of layers, layer widths, dropout rate, learning rate, and weight decay, is selected through a grid search guided by validation performance. This tuning process is performed only for the baseline model and is not re-optimized separately for each architectural variant introduced later.

The final baseline model contains 179,201 trainable parameters. This parameter count is reported explicitly and is used as a reference in later sections when comparing different training and fine-tuning strategies that vary in the number of trainable parameters.

4 Feature Grouping and Encoding Strategies

4.1 Hypothesis and Motivation

Tabular datasets often combine features with distinct semantic roles, scales, and statistical properties. In the movie metadata setting, attributes such as budget and runtime differ qualitatively from identifiers such as director or production company. A standard shared-encoding architecture treats all inputs uniformly, allowing interactions between all features from the earliest layers, but does not explicitly exploit this semantic structure.

We hypothesize that explicitly grouping features according to their semantic meaning and processing each group with a dedicated encoder may lead to more informative intermediate representations. Under this hypothesis, group-specific encoders can first capture intra-group patterns, while a subsequent fusion stage can integrate information across groups. This design introduces a human-defined inductive bias based on domain knowledge, rather than learning the grouping automatically.

4.2 Feature Groups

Based on the structure of the dataset and domain intuition, we manually partition the input features into three semantic groups:

- **Financial features**, including attributes related to production scale and market exposure (e.g., budget and number of votes).
- **Creative features**, including categorical descriptors associated with creative decisions and personnel, such as genre, director, and main actor.
- **Metadata features**, including auxiliary descriptive attributes such as release year, rating, and country.

This grouping is fixed *a priori* and remains unchanged across all experiments. The goal is not to learn the grouping itself, but to study the effect of enforcing such structure within the model architecture.

4.3 Model Variants

To evaluate the effect of feature grouping and fusion, we implement several architectural variants that differ in how feature groups are processed and combined. All variants use the same baseline architecture and training setup unless stated otherwise.

4.3.1 Single-Group Models

As a preliminary step, we train one model per feature group, where each model receives only the features belonging to a single group. Each single-group model uses the same architecture as the baseline, restricted to its corresponding input features.

The purpose of these models is not to achieve optimal performance, but to assess the standalone predictive signal contained within each feature group and to provide reference points for later fusion strategies.

4.3.2 Fixed and Learned Output Fusion

We evaluate fusion strategies that combine predictions from the three single-group models at the output level. In fixed ensemble variants, predictions are aggregated without additional learning, using either simple averaging or manually weighted averaging based on validation performance. In both cases, the base models are trained independently and remain frozen during aggregation.

In learned output fusion, the three scalar predictions are concatenated and passed through a single fully connected neuron to produce the final output. We evaluate both frozen-base and end-to-end training regimes, allowing us to isolate the effect of learning fusion weights while restricting cross-group interaction to the output layer.

4.3.3 Feature-Level Fusion

Feature-level fusion combines intermediate representations rather than final predictions. We extract the 64-dimensional output of the last hidden layer from each group-specific encoder and concatenate them into a 192-dimensional vector. This representation is processed using either a linear mapping ($192 \to 1$) or a small nonlinear MLP ($192 \to 96 \to 64 \to 1$).

As before, we evaluate both frozen-base and end-to-end training regimes, controlling whether gradients propagate into the group-specific encoders.

Figure 1 summarizes the fusion architectures evaluated in this study.

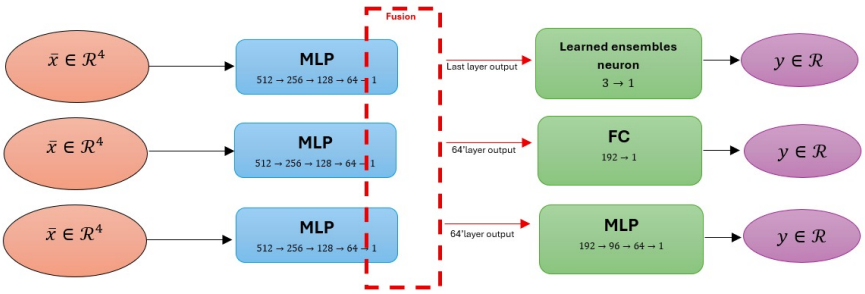

Figure 1: Fusion variants for feature-grouped models.

5 Experiments and Results: Feature Encoding Study

5.1 Experimental Setup

To evaluate the effect of feature grouping and fusion strategies, we compare the architectural variants described in Section 4 under a controlled experimental setting. All models are trained and evaluated on the same dataset splits described in Section 2, using identical optimization procedures unless stated otherwise. This ensures that observed differences in performance can be attributed to architectural design rather than training inconsistencies.

The baseline model processes all features jointly through a shared encoder. In contrast, the feature-grouped variants operate on three predefined semantic groups and differ only in how representations from these groups are combined. For fusion-based models, we evaluate both frozen and end-to-end training regimes, allowing us to isolate the impact of gradient flow across feature groups.

Model performance is evaluated using mean squared error (MSE) on the held-out test set. Validation performance is used solely for model selection and early stopping.

5.2 Quantitative Results

Figure 2 reports test-set MSE for the baseline model and the feature-grouped variants. Single-group models achieve substantially worse performance than the baseline, indicating that no individual feature group is sufficient to capture the full predictive structure of the task.

Fixed ensemble methods, including simple averaging and manually weighted averaging, improve over single-group models but remain clearly inferior to the baseline. Learned output fusion provides a modest improvement over fixed ensembles, particularly when trained end-to-end, but still does not fully close the gap to the shared-encoder baseline.

Feature-level fusion variants exhibit a similar pattern. When the group-specific encoders are frozen and only the fusion module is trained, performance remains limited. Allowing end-to-end training improves results, but even in this setting, fusion-based architectures do not consistently outperform the baseline model.

6 Conclusion: Feature Grouping Study

Intuitively, our architectural design was motivated by an analogy to Convolutional Neural Networks. In CNNs, restricting early receptive fields is beneficial because visual data exhibits strong local correlations: early layers capture simple local patterns, which are later composed into global representations. By analogy, we hypothesized that separating tabular features into semantically related groups would allow the model to first learn group-specific representations and then integrate them at a later fusion stage.

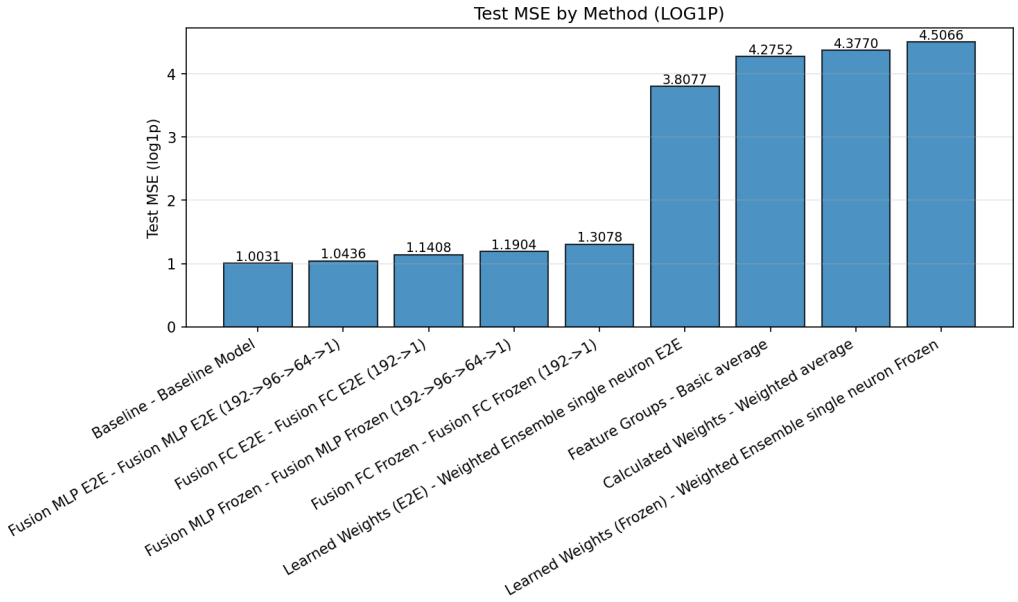

Figure 2: Test-set MSE for the baseline model and feature-grouped variants under different fusion strategies and training regimes.

This intuition does not hold in our dataset. Unlike images, the movie metadata features we consider do not exhibit a meaningful notion of spatial locality, and many of the semantic dependencies relevant to prediction are global and immediate. By separating feature groups, we restricted the effective receptive field of early layers, preventing neurons from accessing cross-group information until late fusion and thereby delaying or suppressing important interactions.

This effect can be formalized from a representational perspective. A sufficiently expressive neural network that processes the full input vector $(x_1, \ldots, x_{12})$ can approximate arbitrary continuous functions over the input space, enabling it to model interactions such as those between *Budget* and *Director*. In contrast, single-neuron fusion models, where the output is defined as

$$\alpha f_1(x_1, \ldots, x_4) + \beta f_2(x_5, \ldots, x_8) + \gamma f_3(x_9, \ldots, x_{12}), \quad \alpha + \beta + \gamma = 1,$$

are restricted to linear combinations of functions defined on disjoint feature subsets and therefore lack universal approximation capacity over the full input space.

While end-to-end training allows gradients to partially compensate for this restriction by adapting earlier representations, frozen training blocks such adaptation and leads to inferior performance.

Thus, we arrive at a key conclusion: The model did not fail to learn the interactions — it was never allowed to represent them.

7 Pretraining and Fine-Tuning Under Temporal Shift

7.1 Motivation and Hypotheses

Movie industry data is inherently non-stationary: production scales, audience preferences, and market dynamics evolve over time. As a result, models trained on historical data may face a distribution shift when applied to movies released in later years. Rather than treating this shift as noise, we explicitly study it as a controlled adaptation problem.

Within this setting, we investigate how different fine-tuning strategies trade off adaptation capacity, data efficiency, and optimization stability. In particular, we formulate the following hypotheses:

- **H1:** Allowing a larger number of parameters to adapt improves generalization under temporal distribution shift, provided sufficient data and training budget are available. Under this hypothesis, full fine-tuning is expected to achieve the lowest test error in high-budget regimes.
- **H2:** Under constrained optimization budgets, parameter-efficient fine-tuning methods may match or outperform full fine-tuning, as they are easier to optimize and less prone to instability when only a small number of gradient updates is allowed.

These hypotheses guide the design of the experiments described in the following subsections.

7.2 Pretraining Setup and Temporal Split

To study adaptation under temporal distribution shift, we adopt a pretrain–fine-tune framework. A baseline model with the architecture described in Section 3 is first trained on movies released up to the year 2005, which constitutes approximately 60% of the filtered dataset. This pretraining phase provides a common initialization that captures patterns present in historical data.

The pretrained model is then adapted using movies released from 2005 onward. This chronological split reflects a realistic deployment scenario, in which models trained on past data must generalize to future samples whose statistical properties may differ. All fine-tuning variants described later start from the same pretrained weights, ensuring that observed performance differences arise from the adaptation strategy rather than differences in initial representations.

7.3 Zero-Shot Evaluation

Before performing any fine-tuning, we evaluate the pretrained model in a zero-shot setting by directly applying it to post-2005 movies without updating its parameters. Performance is reported using mean squared error (MSE), aggregated by release year.

This evaluation serves two purposes. First, it quantifies the severity of the temporal distribution shift by revealing how predictive performance degrades as the temporal distance from the pretraining period increases. Second, it establishes a reference point against which the effectiveness of all subsequent fine-tuning strategies can be assessed.

7.4 Fine-Tuning Strategies

We evaluate several fine-tuning strategies that differ in the scope and structure of parameter updates applied to the pretrained model:

- **Full fine-tuning**, in which all parameters of the pretrained model are updated during adaptation.
- **Gradual unfreezing**, where training begins with only the final layer unfrozen, and earlier layers are progressively released during fine-tuning.
- **Head-only fine-tuning**, which restricts training to the final regression layer while keeping the pretrained encoder fixed.
- **Parameter-efficient fine-tuning methods**, including LoRA and DoRA, which introduce additional low-rank adaptation parameters while freezing the original model weights.

These strategies span a wide range of adaptation capacities, from minimal parameter updates to full model retraining.

7.5 Hyperparameter Selection and Trainable Parameter Counts

For each fine-tuning strategy, we perform a variant-specific mini grid search to select appropriate hyperparameters, such as learning rate and method-specific parameters (e.g., rank for LoRA and DoRA). Hyperparameters are selected based on validation-set performance and then fixed for all subsequent experiments involving that strategy.

Table 1 summarizes the number of parameters updated by each fine-tuning strategy.

Table 1: Number of trainable parameters for different fine-tuning strategies.

Fine-tuning strategy	**Trainable parameters**
Full fine-tuning	179,201
Gradual unfreezing	$65 \rightarrow 179{,}201$
Head-only fine-tuning	65
LoRA	15,464
DoRA	8,693

8 Experiments and Results: Fine-Tuning Study

8.1 Zero-Shot Evaluation

We first evaluate the pretrained model on post-2005 movies in a zero-shot setting, without any fine-tuning. This experiment quantifies the effect of temporal distribution shift when a model trained on historical data is applied to future samples.

Figure 3 shows zero-shot test MSE aggregated by release year. Performance degrades as the temporal distance from the pretraining period increases, indicating a clear mismatch between training and evaluation distributions. This result motivates the need for adaptation and provides a reference point for subsequent fine-tuning experiments.

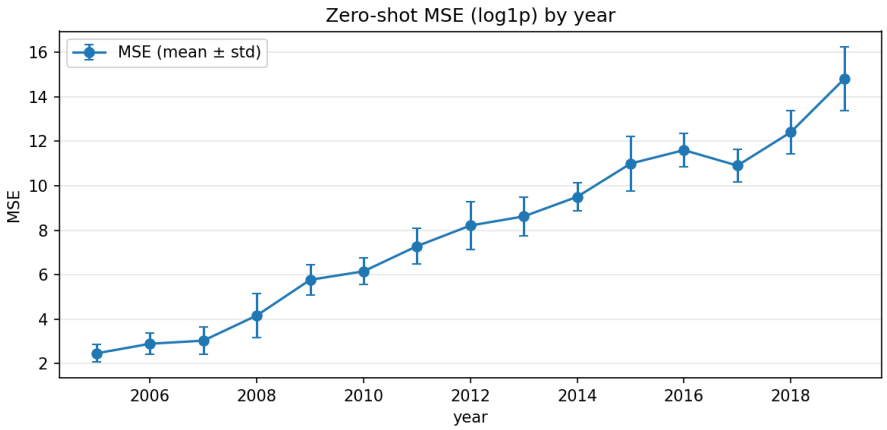

Figure 3: Zero-shot test MSE aggregated by release year for the pretrained model evaluated on post-2005 movies.

8.2 Budget-Constrained Fine-Tuning

We evaluate fine-tuning performance under constrained optimization budgets by limiting the number of gradient updates. Full fine-tuning, gradual unfreezing, head-only fine-tuning, and parameter-efficient methods (LoRA and DoRA) are compared under identical conditions.

Figure 4 reports test-set MSE as a function of the update budget. Methods that allow representation adaptation improve consistently with additional updates, while head-only fine-tuning shows limited sensitivity to budget increases. Parameter-efficient methods achieve strong performance in the low-update regime, often approaching full fine-tuning despite updating far fewer parameters, consistent with Hypothesis H2.

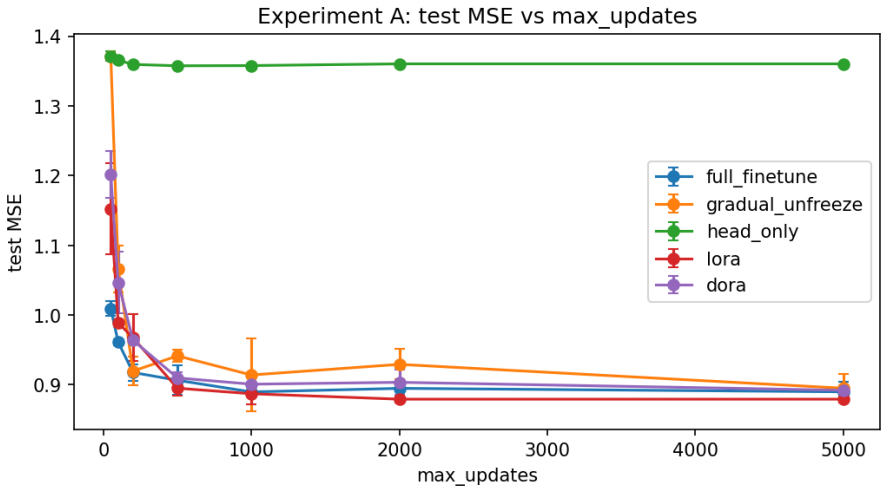

Figure 4: Test-set MSE as a function of the number of gradient updates for different fine-tuning strategies.

8.3 Data-Limited Fine-Tuning

To study the effect of data availability, we fine-tune models using progressively smaller fractions of the post-2005 dataset. Figure 5 shows test-set MSE as a function of the available data fraction.

Most methods benefit from increased data, with performance gaps narrowing at larger fractions. Head-only fine-tuning remains largely insensitive to data size and consistently underperforms, indicating a performance ceiling imposed by restricted adaptation capacity.

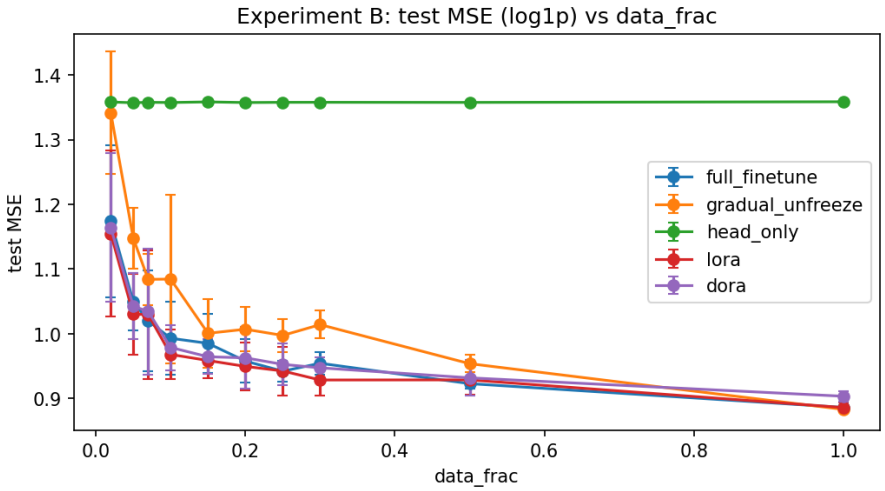

Figure 5: Test-set MSE as a function of the fraction of available fine-tuning data.

8.4 Compute-Aware Comparison

The comparisons above do not account for large differences in the number of trainable parameters across methods. To enable a fairer assessment, we analyze performance as a function of a pseudo-compute budget, defined as the product of the number of trainable parameters and the number of gradient updates.

Figure 6 shows test-set MSE as a function of this pseudo-compute measure. Parameter-efficient methods such as LoRA and DoRA achieve competitive performance at substantially lower compute budgets than full fine-tuning. Full fine-tuning reaches similar error levels only at much higher pseudo-compute values, while gradual unfreezing occupies an intermediate regime. Head-only fine-tuning consistently underperforms across the compute spectrum.

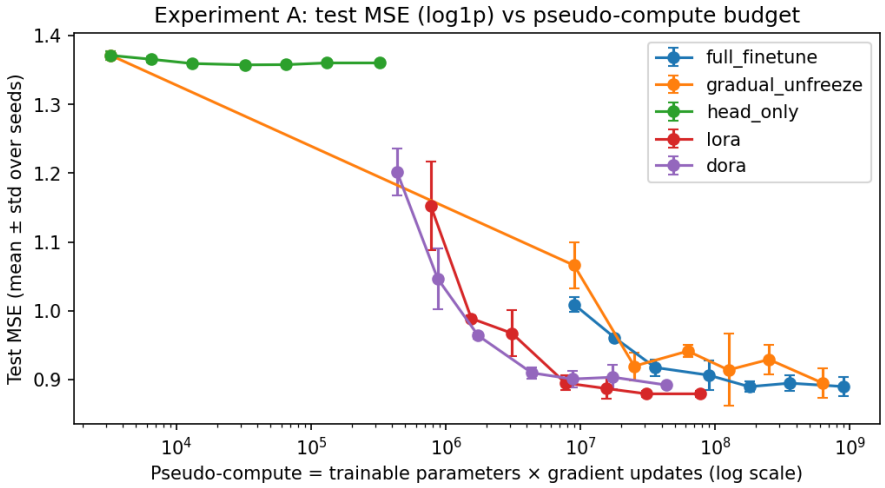

Figure 6: Test-set MSE as a function of pseudo-compute budget (trainable parameters × gradient updates).

9 Conclusion: Fine-Tuning Study

By comparing full fine-tuning, gradual unfreezing, head-only adaptation, and parameter-efficient methods (LoRA and DoRA) in a controlled pretrain–fine-tune setup, we show that fine-tuning performance cannot be interpreted independently of the number of parameters allowed to adapt. Methods that achieve similar test error may differ by orders of magnitude in trainable parameter count, leading to substantially different computational costs.

A central observation is that LoRA and DoRA achieve performance comparable to full fine-tuning while updating far fewer parameters. This suggests that the adaptation required to move from the historical (pre-2005) distribution to the modern one lies in a low-dimensional subspace of the pretrained parameter space. By freezing the backbone and learning structured low-rank updates, these methods enable effective adaptation

with significantly reduced computational and memory overhead. DoRA further improves optimization stability by decoupling the magnitude and direction of updates without increasing adaptation capacity.

In contrast, head-only fine-tuning consistently underperforms across all optimization budgets. Freezing the entire backbone restricts the hypothesis space to representations learned during pretraining, inducing a dominant approximation error under temporal distribution shift. As a result, head-only adaptation exhibits a clear performance ceiling and remains largely insensitive to increased optimization budget.

The importance of accounting for trainable parameter count becomes especially evident in low-budget regimes. When results are analyzed through a compute-aligned perspective, the apparent advantage of full fine-tuning diminishes, while parameter-efficient methods achieve comparable performance at much lower effective cost. These findings demonstrate that any fair evaluation of fine-tuning strategies must explicitly consider trainable parameter count, rather than relying on raw accuracy alone.

Ethics Statement

Introduction

Student names: Yuval Dreizin, Tomer Ohayon
Project title: *"Gross-Ahead": Predicting Movie Gross Revenue using Deep Learning*

This project studies the use of deep learning models for predicting movie gross revenue based on historical tabular data, including financial, creative, and metadata features. The goal is to analyze architectural design choices, feature interaction assumptions, and fine-tuning strategies under temporal distribution shift, with an emphasis on generalization and robustness rather than deployment.

Large Language Model (LLM, ChatGPT) Analysis

Stakeholders

- Machine learning researchers and students.
- Industry practitioners using predictive models.
- Decision-makers in the film or media industry.

Explanation to Stakeholders

Stakeholders would be informed that the model is intended for research purposes only and produces approximate statistical predictions rather than reliable forecasts. Its limitations—such as sensitivity to historical biases, temporal shifts, and missing real-world factors—would be clearly stated, and predictions should not be used as the sole basis for real-world decisions.

Responsibility

The responsibility for providing this explanation lies with the researchers and developers who design, train, and present the model.

Reflection on the AI Output (Independent, Creative Thinking)

A common ethical concern with AI-generated responses is that they may appear fluent and authoritative even when the underlying information is uncertain or incomplete, potentially leading to overconfidence or misuse. However, in this case, the AI's response explicitly acknowledges the limitations of the model, including historical bias, temporal distribution shift, and the risk of misinterpreting predictions as reliable forecasts. By clearly framing the model as a research tool and cautioning against its use for real-world decision-making, the response actively addresses the ethical risk associated with misleading or overconfident AI outputs. This demonstrates that transparent communication of uncertainty is an effective way to mitigate ethical concerns arising from AI-generated explanations.